



AAI presents IUT 12th ICT Fest
powered by Therap (BD) Ltd.
organized by IUT Computer Society

Bdapps presents
Agentic AI Hackathon
powered by Codex

Final Round Problem Statement

AgriSense AI

*Build an agent that takes a farmer from an empty field to a costed, weather-aware season plan,
and keeps advising through harvest.*

The Problem

A smallholder farmer preparing for a new season has to answer a chain of decisions where each choice constrains the next: which crop to plant, when to sow, how much fertilizer and water to budget, what pests to expect, what the harvest will cost, and whether the numbers leave any profit. The information needed to answer these questions already exists. It sits in weather services, soil guidelines, fertilizer charts, extension manuals, and market price boards. It is scattered across sources the farmer cannot easily reach, in formats that assume a specialist reader, and it arrives too late to change a planting decision.

The result is that many planting and input decisions are made from memory and habit. A wrong crop choice for the soil and season, fertilizer applied right before heavy rain, or selling at the season's price floor each turns into losses the farmer only sees at settlement.

This is a strong fit for an AI agent rather than a search box or a static calculator. Doing the job well requires holding a conversation to gather the farm's specifics, pulling live external data, running the arithmetic, sequencing steps over a whole season, and explaining each recommendation in terms the farmer can act on and trust.

Your Challenge

Build an autonomous agricultural advisor that plans, reasons, and acts across a growing season for a specific farm. It should talk to the farmer to learn their situation, retrieve real external data, compute a costed plan, and explain its reasoning at every step.

The bar is an **agent, not a chatbot**. A chatbot answers the question in front of it. Your agent should gather what it is missing, decide which tools to call, chain several steps toward a goal, remember what it already learned, and adapt when conditions change.

What “agentic” means for this challenge

Judges will look for these behaviors specifically. Aim to demonstrate all five in the core build:

1. **Tool use:** the agent calls at least one real external service (for example, a weather API) and uses the returned values in its output rather than inventing them.
2. **Multi-step planning:** a single farmer request triggers a sequence of dependent steps, not one lookup.
3. **Handling missing information:** when the farmer's input is incomplete, the agent identifies the specific gaps and asks targeted follow-ups instead of guessing or failing.
4. **Memory:** the agent carries context across turns (and ideally across sessions) so the farmer never repeats themselves.
5. **Explainability:** every recommendation names the inputs behind it. “Apply 45 kg/acre urea in the next 3 days, because your soil is sandy, rice is at the vegetative stage, and no rain is forecast this week” beats “Apply urea.”

Scope: Build in Tiers

You have 24 hours. Ship one complete feature before you add a second. A working core that runs end to end beats ten half-built features in a demo, and scope control is one of the things judges weigh most heavily.

Treat **Tier 0 as required**, Tier 1 as where strong teams differentiate, and Tier 2 as a bonus that only counts if Tier 0 is solid.

Tier 0: Core (required)

The core is a single path: **from a short conversation, produce a grounded, explained, costed season plan for one farm.** All of the following should work end to end.

#	Capability	Done when
1	Conversational intake	From a vague opening message, the agent collects at minimum: location, farm size, soil type, water availability, budget, and target season. It asks targeted follow-ups only for the fields still missing.
2	Live weather grounding	The agent calls a real weather API using the farm's location and uses the actual returned values (rainfall, temperature) in its recommendations. No invented forecasts.
3	Crop recommendation	The agent ranks at least 3 candidate crops for the profile, season, and weather, each with suitability, water need, risk level, and a rough profit estimate.
4	Season plan	For the chosen crop, the agent produces a dated calendar from land preparation to harvest (sowing window, fertilizer timing, irrigation, weed and pest checkpoints, harvest).
5	Financial projection	An itemized cost breakdown plus expected yield, revenue, net profit, ROI, and break-even. The math is inspectable and internally consistent: change an input, and the outputs change correctly.
6	Explained reasoning	Every recommendation states the specific farm inputs and retrieved data it rests on.
7	Knowledge base with RAG	Relevant agronomic data is collected from publicly available sources (extension manuals, fertilizer guides, crop calendars, soil and yield references) and built into a knowledge base. The agent retrieves from it, and the crop, fertilizer, and season-plan advice is grounded in what it retrieves rather than in model recall alone.
8	Visible agent trace	The interface exposes a trace of the agent's work: every tool call, the parameters sent, and the raw values returned; either in a side panel, an expandable section, or a log view. A judge must be able to confirm that a number in the plan came from a real call and not from the model's imagination.

If you build only Tier 0 and it runs cleanly in the demo, you have a complete, competitive submission.

Tier 1: Advanced (where strong teams pull ahead)

- **Persistent memory:** the agent remembers the farm and prior conversations across sessions, not just within one chat.
- **Proactive, weather-triggered advice:** the agent watches the forecast and adjusts the plan. Example: “Heavy rain in 4 days. Delay the nitrogen application by 4 days to cut runoff loss.”
- **Fertilizer and irrigation scheduler:** quantities, timing by growth stage, organic alternatives, and cost, tied to the specific crop and soil.
- **Pest and disease risk:** predict likely pests and diseases from crop, growth stage, and weather, with preventive and treatment options and estimated cost.
- **Scenario simulation:** the farmer asks “what if rainfall drops 30%?” or “what if my budget is cut 40%?” and the agent returns a revised plan with changed numbers, not a generic answer.

Tier 2: Ambitious (bonus, only after Tier 0 is solid)

- **Marketplace and supplier comparison:** match input needs to suppliers ranked by price, delivery time, distance, and rating. A seeded or mock supplier catalog is completely acceptable.
- **Market price intelligence:** current and historical prices, plus a sell-now/store/wait recommendation with reasoning.
- **Plant disease detection from images:** let the farmer upload a leaf photo and classify the disease.
- **bdapps Payment Gateway Integration:** integrate the official bdapps CaaS (Charging as a Service) API in sandbox/simulator mode. Showcase a complete checkout, operator balance deduction, and receipt flow. Follow this documentation: https://dev.bdapps.com/API_Documentation/bdapps_tap_api.html
- **Bengali or voice interaction** for low-literacy accessibility.

What Not to Do

The original brief for this problem listed fourteen feature areas including live escrow payments and a full marketplace. Do not attempt that in 24 hours. Building all of it halfway is the most common way teams lose. Pick the core, make it real, and add tiers only when the layer beneath them works. If you demo a payment gateway but your crop recommendation ignores the weather it just fetched, judges will notice.

Location & Timeline

Location: In-person at IUT. (Room 301 and 302, 2nd Academic Building)

Milestone	When
Check-in & opening briefing	24th July 08:00 am
Hacking begins (clock starts)	24th July 09:00 am
Hacking ends (clock ends)	25th July 09:00 am
Demos & judging	25th July 10:00 am

Allowed Resources

You **may** use:

- **Open-source libraries and frameworks** — any agent framework, RAG toolkit, or library with a compatible license.
- **Paid and third-party APIs** — weather, LLM providers, hosted vector databases, and similar. Teams provide their own keys and cover their own costs.
- **LLM coding assistants** — Claude Code, Copilot, Cursor, and similar AI pair-programmers, for writing and debugging code during the event.

You **must not** use:

- **Pre-built project code.** All application code must be written during the 24-hour window. Publicly available scaffolding generated at the start (framework CLIs, starter templates) is fine, but bringing a pre-existing AgriSense-style codebase is not.

Whatever you use, your README must clearly state what is real versus mock, as required in the submission.

Submission Requirements

To be judged, submit:

1. **A working prototype** in GitHub/GitLab with source code including a README covering setup, the tools and APIs used, and which tier each feature reaches. Also include which data is collected/real and what is generated/mock.
2. **A 4-minute live demo**, followed by 2–3 minutes of Q&A.

Submission Mechanics

- **Where:** The submission channel will be shared during the event.
- **What:** A GitHub/GitLab repository link with the source code and README described above. The live demo is delivered in person during judging.
- **Naming:** Name your repository `TeamName_AgriSense` (for example, `TeamAgro_AgriSense`).
- **On time:** A submission counts as on time only if the final commit is pushed before the hard cutoff. Commits after the cutoff are not considered. Submit your repo link before the deadline even if you keep polishing locally.

Judging Criteria

Scored out of 100. Don't spend too much time on UI/UX.

Criterion	Points	What we are looking for
Agentic behavior	20	Real tool use, multi-step planning, recovery from missing information, and memory. Not a single-shot chatbot.
Scope and execution	15	A core with proper scope that runs end to end and is stable in the demo.
Accuracy and practicality	20	Recommendations a real farmer could act on. Correct financial math. Grounded in the retrieved data.
Knowledge base	12	Relevant data collected from publicly available sources, built into a usable knowledge base, with RAG implemented on top of it and actually feeding the agent's advice.
bdapps Payment Gateway Integration	10	Successful simulation of the bdapps CaaS API flow (request/response) during checkout.
Explainability	10	Recommendations state their reasoning and inputs, and the farmer could reasonably trust them.
Technical implementation	8	Clean integration of external APIs and data, sound architecture, working tests where they matter.
Innovation	5	Genuinely useful additions beyond the core: scenario simulation, proactive alerts, image diagnosis, marketplace. Novelty only counts on top of a working core.
Total	100	

Code of Conduct

We want a welcoming, professional event for everyone. All participants, mentors, and organizers agree to:

- **Treat everyone with respect.** No harassment, discrimination, or demeaning behavior based on gender, ethnicity, religion, disability, or any personal characteristic — in person, in chat, or on stage.
- **Keep it your own work.** Submit what your team builds during the event. Using allowed libraries and APIs is expected; passing off someone else's project as your own is not.
- **Compete fairly and safely.** No sabotage of other teams, no tampering with shared infrastructure, and follow all venue and safety rules.
- Harassment, cheating, or repeated disruption can lead to a warning, disqualification, or removal from the venue at the organizers' discretion.

Good luck.